

Universidad Nacional de Río Cuarto
Facultad de Ciencias Exactas, Físico-Químicas y Naturales
Departamento de Computación
Licenciatura en Ciencias de la Computación

Bases de Datos II

Resumen Teórico

DCL — Lenguaje de Control de Datos
Práctico N.º 2

Alumno: Tomás Rodeghiero
Año: 2026

Índice

1. Introducción	2
2. Seguridad: el problema y los mecanismos	2
3. Asignación de privilegios: GRANT	3
3.1. Sintaxis general	3
3.2. Privilegios habituales	3
3.3. Ejemplos básicos del teórico	3
4. Eliminación de privilegios: REVOKE	4
5. Propietarios y permisos sobre el esquema	4
6. Grafo de concesión de permisos	4
7. Roles	5
8. Diferencias entre motores	6
8.1. MySQL	6
8.2. PostgreSQL	6
8.3. Oracle XE	7
9. Administración: usuarios, contraseñas y herramientas	8
10. Conexión con los ejercicios del práctico	9
11. Verificación de privilegios	11
12. Ideas clave para repasar antes del parcial	11
Resumen final	12

1. Introducción

El **DCL** (*Data Control Language*) es la parte del lenguaje SQL dedicada a la **seguridad** y al **control de acceso** a los recursos de la base de datos. Mientras que el DDL define las estructuras (tablas, vistas, dominios) y el DML manipula los datos, el DCL responde a una pregunta distinta: *¿quién puede hacer qué sobre cada objeto?*

Una base de datos en producción rara vez se accede con un único usuario. Se necesita distinguir, por ejemplo, entre un *cajero* que sólo puede consultar y modificar ciertas tablas, un *administrativo* con permisos más amplios y un *DBA* que controla la base completa. Para sostener ese tipo de organización SQL provee tres mecanismos complementarios:

- **Usuarios:** cuentas con las que se inicia sesión en el motor.
- **Privilegios:** permisos individuales sobre objetos (tablas, vistas, esquemas, procedimientos, etc.).
- **Roles** (o *grupos*): conjuntos nominados de privilegios que se asignan a uno o varios usuarios para evitar repetir trabajo de configuración.

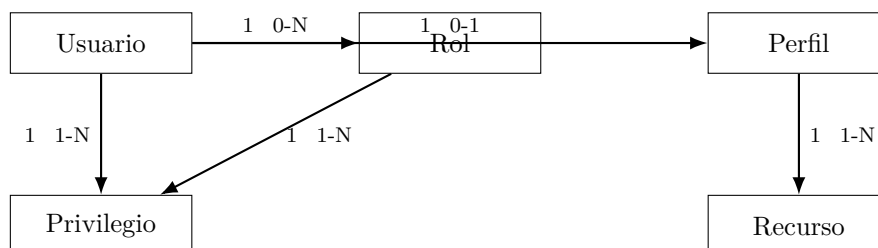
Idea clave del Teórico 3. El estándar SQL define los conceptos de *usuario*, *rol* y *privilegio*, y los comandos **GRANT** / **REVOKE**. Sin embargo, *la administración real de usuarios la implementa cada motor a su manera*: MySQL, PostgreSQL y Oracle XE no son compatibles entre sí en este punto. Reconocer qué es estándar y qué es específico es la mitad del práctico.

2. Seguridad: el problema y los mecanismos

SQL ofrece la siguiente caja de herramientas para proteger los datos:

1. **Autenticación:** definición de usuarios y roles, con sus contraseñas y, en algunos motores, restricción del *host* desde el que se aceptan las conexiones.
2. **Autorización:** asignación de privilegios sobre objetos. Sin un privilegio explícito un usuario no puede leer ni escribir nada que no le pertenezca.
3. **Control de recursos:** límites de uso (cantidad de consultas por hora, tiempo de inactividad, número máximo de conexiones simultáneas) que evitan abusos.
4. **Trazabilidad implícita:** el grafo de concesiones permite, en cualquier momento, contestar quién dio qué privilegio a quién.

Modelo conceptual. El teórico cierra con un diagrama que sintetiza las relaciones entre los conceptos:



Un *usuario* se asocia con uno o más *privilegios* (directamente o a través de *roles*) y, opcionalmente, con un *perfil* que regula su consumo de *recursos*.

3. Asignación de privilegios: GRANT

3.1. Sintaxis general

El comando GRANT concede privilegios sobre objetos a usuarios o roles:

```
GRANT {ALL | listaPrivilegios}
      ON TABLE listaTablas
      TO listaUsuariosRoles
[WITH GRANT OPTION];
```

Significado de los componentes:

- ALL: otorga *todos* los privilegios soportados sobre el objeto. Es cómodo, pero peligroso.
- listaPrivilegios: una o más operaciones permitidas. Las clásicas son SELECT, INSERT, UPDATE, DELETE, REFERENCES y EXECUTE.
- listaTablas: el objeto (o lista de objetos) sobre el que se aplican; en el estándar es uno solo, varios motores aceptan listas o comodines (*.*, db.*).
- listaUsuariosRoles: a quién se le da el permiso.
- WITH GRANT OPTION: el receptor podrá, a su vez, delegar ese mismo permiso a un tercero.

3.2. Privilegios habituales

Privilegio	Permite...
SELECT	Leer filas de una tabla o vista.
INSERT	Agregar filas.
UPDATE	Modificar filas existentes (puede acotarse a ciertas columnas).
DELETE	Eliminar filas.
REFERENCES	Crear claves foráneas y CHECKs que referencien la tabla/-columna.
EXECUTE	Ejecutar funciones o procedimientos almacenados.
CREATE, DROP, ALTER	Operar sobre la estructura de bases, tablas o índices.
INDEX	Crear y eliminar índices.
CREATE VIEW, SHOW VIEW	Trabajar con vistas.
CREATE ROUTINE, ALTER ROUTINE	Crear/modificar procedimientos almacenados.
GRANT OPTION	Delegar privilegios a otros usuarios.

Granularidad por columna. Para SELECT, INSERT, UPDATE y REFERENCES es posible especificar la lista de columnas a las que aplica el permiso, escribiéndolas entre paréntesis después del nombre del privilegio: GRANT SELECT(apellido, nombre) ON personas TO U1. Es la forma natural de cumplir consignas del práctico como “*puede consultar los campos apellido y nombre de cliente*”.

3.3. Ejemplos básicos del teórico

```
-- 1) SELECT sobre una tabla a varios usuarios.
GRANT SELECT ON personas TO U1, U2;

-- 2) Permitir a U1 crear FKs hacia personas(dni).
GRANT REFERENCES (dni) ON personas TO U1;

-- 3) U1 podrá, a su vez, delegar el SELECT.
```

```
GRANT SELECT ON personas TO U1 WITH GRANT OPTION;

-- 4) SELECT solo sobre dos columnas (PostgreSQL >= 9).
GRANT SELECT (apellido, nombre) ON personas TO U1;
```

4. Eliminación de privilegios: REVOKE

REVOKE es la operación inversa: retira privilegios previamente otorgados.

```
REVOKE {ALL | listaPrivilegios}
      ON listaTablas
      FROM listaUsuarios [RESTRICT | CASCADE];
```

RESTRICT vs CASCADE. Cuando un usuario otorgó un permiso (vía `WITH GRANT OPTION`) a otros, al revocárselo hay que decidir qué hacer con esos terceros:

- **CASCADE:** el revoke se propaga. Si U1 dio el permiso a U3 y a U1 se le revoca, U3 también lo pierde.
- **RESTRICT:** si revocar implicaría romper permisos transferidos, la operación falla. Permite preguntar antes de propagar.

```
REVOKE SELECT ON personas FROM U1 RESTRICT;
```

Por defecto, REVOKE es en cascada en el estándar. PostgreSQL implementa correctamente la diferencia `RESTRICT/CASCADE`; **MySQL no la respeta**, y revoca de forma directa sin distinguir explícitamente.

5. Propietarios y permisos sobre el esquema

Propiedad. El *propietario* de un objeto es siempre el usuario que lo creó. Sólo el propietario (o un usuario con privilegios administrativos suficientes) puede otorgar permisos sobre el objeto a terceros. De ahí que en el práctico, antes de hacer cualquier `GRANT`, se conecte el DBA o el dueño del esquema.

Operaciones de esquema. Las operaciones que modifican la *estructura* de la base (`CREATE`, `DROP`, `ALTER`) sólo pueden ser realizadas por el propietario del esquema o por usuarios con los privilegios correspondientes (`CREATE`, `DROP`, `ALTER` sobre la base).

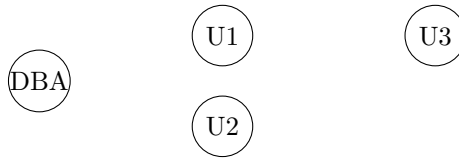
6. Grafo de concesión de permisos

Una herramienta conceptual muy práctica para razonar sobre el estado de los permisos es el **grafo de concesión**: un grafo dirigido *por recurso y tipo de privilegio* en el que los nodos son los usuarios y las aristas representan concesiones.

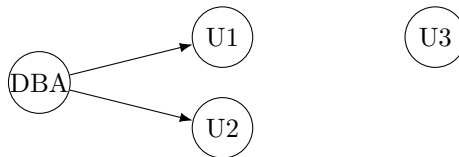
- Existe un arco $U_i \rightarrow U_j$ si U_i concedió a U_j el privilegio en cuestión.
- La raíz es siempre el *DBA* o el *dueño del objeto*.
- Un usuario tiene el permiso si y sólo si existe un camino desde la raíz hasta su nodo.

Esa caracterización en términos de caminos es la que justifica el comportamiento de **REVOKE ... CASCADE**: si al cortar un arco algunos nodos quedan sin camino desde la raíz, automáticamente pierden el privilegio.

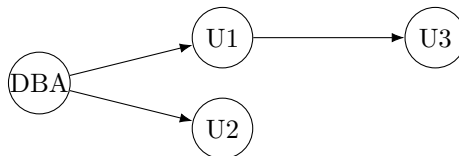
Ejemplo paso a paso (del Teórico 3). (1) **Estado inicial.** El DBA es el único con **UPDATE** sobre **personas**.



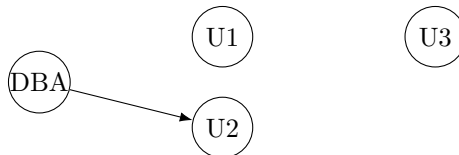
(2) El DBA ejecuta: **GRANT UPDATE ON personas TO U1, U2 WITH GRANT OPTION.**



(3) U1 ejecuta: **GRANT UPDATE ON personas TO U3.**



(4) El DBA ejecuta: **REVOKE UPDATE ON personas FROM U1 CASCADE.** Como el único camino al nodo U3 pasaba por U1, U3 también pierde el privilegio.



7. Roles

Un **rol** representa un *tipo de usuario*: agrupa privilegios asociados a una función dentro de la organización (cajero, asesor, empleado, encargado). En lugar de repetir las mismas concesiones para cada cuenta, se declara un rol una sola vez y se le asigna a las cuentas que lo necesiten. Cuando aparece un nuevo cajero, basta con asignarle el rol *cajero* y queda configurado.

Administración de roles.

```

-- Crear y eliminar roles.
CREATE ROLE nombreRole;
DROP ROLE nombreRole;

-- Conceder un rol a un usuario.
GRANT role TO usuario;

-- Conceder un rol a otro rol (anidamiento).
GRANT role1 TO role2;
  
```

Las dos últimas formas explican por qué **GRANT** aparece dos veces en una secuencia típica de configuración: primero se otorgan los privilegios al rol y luego se otorga el rol a los usuarios.

8. Diferencias entre motores

La administración de usuarios y roles *no es estándar*. Cada motor tiene sus comandos, su modelo de cuentas y sus opciones de seguridad.

8.1. MySQL

MySQL trabaja con *cuentas* de la forma `usuario@host`, donde *host* indica desde qué máquina se acepta la conexión:

- `'app'@'localhost'`: sólo conexiones locales.
- `'app'@' %'`: el comodín `%` acepta cualquier host.
- Un mismo nombre de usuario puede existir con distintos hosts y distintas contraseñas.

Creación y borrado de cuentas.

```
CREATE USER 'usuario'@'host' [IDENTIFIED BY 'password'];  
DROP USER 'usuario'@'host';
```

Roles (desde MySQL 8).

```
CREATE ROLE [IF NOT EXISTS] role [, role] ...;  
GRANT role [, role] ... TO user_or_role [, user_or_role] ...  
[WITH ADMIN OPTION];
```

Límites de recursos. Desde MySQL 5.0 se pueden acotar:

- `MAX_QUERIES_PER_HOUR`
- `MAX_UPDATES_PER_HOUR`
- `MAX_CONNECTIONS_PER_HOUR`
- `MAX_USER_CONNECTIONS` (conexiones simultáneas).

```
-- Forma clasica (hasta 5.7) con GRANT.  
GRANT ALL ON *.* TO 'app'@'localhost' WITH  
    MAX_QUERIES_PER_HOUR 20  
    MAX_UPDATES_PER_HOUR 10  
    MAX_CONNECTIONS_PER_HOUR 5  
    MAX_USER_CONNECTIONS 2;  
  
-- Forma actual (>= 8) con ALTER USER.  
ALTER USER 'app'@'localhost'  
    WITH MAX_QUERIES_PER_HOUR 500  
    MAX_UPDATES_PER_HOUR 100;
```

Para MySQL 8+. Los límites de recursos en `GRANT` fueron desaprobados como mecanismo principal y la cátedra recomienda usar `ALTER USER`. La sintaxis `GRANT ... WITH MAX...` sigue funcionando por compatibilidad.

8.2. PostgreSQL

En PostgreSQL los conceptos de *usuario*, *rol* y *grupo* son lo mismo: todos son *roles*. La diferencia es operativa:

- Un rol con permiso de `LOGIN` se comporta como *usuario*.
- Un rol `NOLOGIN` se comporta como *grupo*. Los usuarios se hacen miembros de él para heredar sus privilegios.

Creación de roles. La sintaxis es muy expresiva:

```
CREATE ROLE nombre [WITH] [option ...];
-- option puede ser, entre otras:
-- SUPERUSER / NOSUPERUSER
-- CREATEDB / NOCREATEDB
-- CREATEROLE / NOCREATEROLE
-- INHERIT / NOINHERIT
-- LOGIN / NOLOGIN
-- CONNECTION LIMIT n
-- {ENCRYPTED / UNENCRYPTED} PASSWORD 'pwd'
-- VALID UNTIL 'timestamp'
-- IN ROLE/IN GROUP rolename, ...
-- ROLE rolename, ...
```

Existe además el atajo `CREATE USER` (que es `CREATE ROLE` con `LOGIN` por defecto) y la utilidad de línea de comandos `createuser`, con flags como `-P` (pide contraseña), `-d` (puede crear bases), `-A` (no puede crear usuarios) y `-h` (host).

Grupos. `CREATE GROUP` es un alias histórico de `CREATE ROLE` sin `LOGIN`:

```
CREATE GROUP groupName [WITH SYSID groupid] [USER username, ...];
ALTER GROUP groupName {ADD | DROP} USER username; -- equivale a GRANT/REVOKE
DROP GROUP groupName;
```

Acceso desde clientes. Por defecto PostgreSQL acepta conexiones sólo desde `localhost`. Para permitir clientes remotos hay que editar `pg_hba.conf`, por ejemplo:

```
# TYPE DATABASE USER CIDR-ADDRESS METHOD
host all all 0.0.0.0/0 md5
```

8.3. Oracle XE

En Oracle XE **crear un usuario equivale a crear un esquema**: el usuario es el dueño de su propio espacio de objetos.

Creación de usuario (ejemplo).

```
CREATE USER usuario
  IDENTIFIED BY usuario
  DEFAULT TABLESPACE users
  QUOTA 10M ON users
  TEMPORARY TABLESPACE temp
  QUOTA 5M ON system
  PASSWORD EXPIRE;
```

`PASSWORD EXPIRE` fuerza al usuario a cambiar la clave en su próximo *login*, recurso muy usado para cuentas recién creadas (es justamente lo que pide el inciso 4 del práctico para *emplead01*).

Roles predefinidos. Oracle XE trae tres roles listos para asignar:

- `CONNECT`: conectarse a la base de datos.
- `RESOURCE`: para desarrolladores; permite crear objetos pero no vistas.
- `DBA`: rol de administrador.

Perfiles (PROFILE). Oracle permite definir *perfiles* que limitan el uso de recursos por usuario y se asignan luego con `ALTER USER`:

```
CREATE PROFILE desarrollador LIMIT
  SESSIONS_PER_USER 1
  IDLE_TIME 30
  CONNECT_TIME 600;
```

Es la forma natural de cumplir consignas como “*el director sólo puede estar conectado un máximo de 20 minutos*” del Ejercicio 4.

Cuentas APEX vs cuentas del motor. Oracle XE distingue las cuentas de la aplicación web APEX (creadas dentro de un *workspace*) de los usuarios del motor de base de datos. **Los usuarios APEX no son usuarios de la base**; sí lo son los esquemas que esos workspaces crean. El DBA del motor es SYS o SYSTEM, y la administración por shell se recomienda con sqlplus.

Comparación rápida entre motores.

Tema	MySQL 8+	PostgreSQL	Oracle XE
Cuenta	usuario@host con password	Role con LOGIN	Usuario equivale a esquema.
Roles	CREATE ROLE (desde 8)	Todo es rol; CREATE ROLE	CREATE ROLE; trae CONNECT/RESOURCE/DBA.
Restricción de host	Parte del nombre de cuenta	pg_hba.conf	Configuración de red separada.
Limites de recursos	MAX_*_PER_HOUR	CONNECTION LIMIT en el rol	CREATE PROFILE + ALTER USER ... PROFILE.
REVOKE con RESTRICT / CASCADE	No respeta realmente la diferencia	Implementado del estándar	Comportamiento estándar.
Forzar cambio de clave	ALTER USER ... PASSWORD EXPIRE	VALID UNTIL	PASSWORD EXPIRE en CREATE o ALTER USER.

9. Administración: usuarios, contraseñas y herramientas

MySQL. La instalación crea el superusuario `root`; desde la versión 5 solicita la contraseña durante la instalación, y desde la 8 en Linux por defecto autentica con el sistema operativo. Las tablas que almacenan usuarios y permisos viven en la base interna `mysql`. Para la administración se recomiendan **MySQL Workbench** (interfaz actual), `phpMyAdmin` (servicios de hosting) o `MySQLAdministrator` (desactualizado).

PostgreSQL. La instalación crea el superusuario `postgres` (con un usuario homónimo a nivel de sistema operativo en Linux). Para conectarse desde aplicaciones cliente con autenticación por base de datos — y no por SO — hay que ajustar `pg_hba.conf`. Herramientas habituales: **PGAdmin IV** y `phpPgAdmin`.

Oracle XE. Se administra por web en <http://localhost:8080/apex> o, recomendado, por consola con `sqlplus`. En 11XE el usuario DBA del motor es `SYS` o `SYSTEM` y el workspace por defecto es `internal`.

10. Conexión con los ejercicios del práctico

El Práctico 2 está diseñado para ejercitar exactamente los conceptos anteriores. Cada ejercicio toma una de las bases de datos del Práctico 1 y agrega usuarios y permisos sobre ella.

Ejercicio 1 (MySQL) — base de Cliente/Producto/Factura

Sobre la base `practico1a` se crean los usuarios `vendedor1`, `vendedor2` y `administrador` con permisos muy diferenciados. La consigna combina:

- INSERT sobre una tabla específica (`vendedor1` sobre `cliente`; `vendedor2` sobre `factura`).
- SELECT acotado a una tabla (`vendedor2` sobre `producto`).
- UPDATE a nivel de **columna** con `WITH GRANT OPTION` (`administrador` sobre `producto.descripcion`).
- DELETE colectivo sobre `cliente` para los tres.
- REVOKE ALL para limpiar al final los permisos del `vendedor2`.

Esqueleto típico.

```
CREATE USER 'vendedor1'@'localhost' IDENTIFIED BY 'pwd';
CREATE USER 'vendedor2'@'localhost' IDENTIFIED BY 'pwd';
CREATE USER 'administrador'@'localhost' IDENTIFIED BY 'pwd';

USE practico1a;

GRANT INSERT ON practico1a.cliente TO 'vendedor1'@'localhost';

GRANT INSERT ON practico1a.factura TO 'vendedor2'@'localhost';
GRANT SELECT ON practico1a.producto TO 'vendedor2'@'localhost';

GRANT UPDATE (descripcion) ON practico1a.producto
  TO 'administrador'@'localhost' WITH GRANT OPTION;

-- DELETE para los tres.
GRANT DELETE ON practico1a.cliente TO
  'vendedor1'@'localhost',
  'vendedor2'@'localhost',
  'administrador'@'localhost';

-- Verificacion y limpieza final.
SHOW GRANTS FOR 'vendedor2'@'localhost';
REVOKE ALL PRIVILEGES, GRANT OPTION FROM 'vendedor2'@'localhost';
```

Ejercicio 2 (MySQL) — base de Vehículos/Estacionamiento

Aquí entra en juego la dimensión `host`: el `encargado_estacionamiento` se conecta sólo desde `localhost` y el `cobrador` desde cualquier máquina (%). El cobrador, además, queda limitado en recursos con `MAX_CONNECTIONS_PER_HOUR=3` y `MAX_QUERIES_PER_HOUR=10`.

```
CREATE USER 'encargado_estacionamiento'@'localhost' IDENTIFIED BY 'pwd';
CREATE USER 'cobrador'@'%' IDENTIFIED BY 'pwd';

GRANT SELECT, INSERT ON practico1b.parquimetro
  TO 'encargado_estacionamiento'@'localhost';
GRANT SELECT ON practico1b.*
```

```

    TO 'encargado_estacionamiento'@'localhost';

GRANT SELECT ON practico1b.estacionamiento TO 'cobrador'@'%';

ALTER USER 'cobrador'@'%' WITH
    MAX_CONNECTIONS_PER_HOUR 3
    MAX_QUERIES_PER_HOUR 10;

-- Inciso d).
REVOKE SELECT ON practico1b.estacionamiento FROM 'cobrador'@'%';

```

Ejercicio 3 (PostgreSQL) — base de Accidentes

Es el ejercicio que introduce **roles compartidos**. El rol *empleado* consolida lo común a todos (SELECT sobre cliente(apellido, nombre)); a partir de ahí cada usuario recibe los permisos específicos:

- *asesor*: SELECT en *taller* e INSERT en *accidente*.
- *administrativo*: SELECT en *automóvil*.
- *encargado*: DROP sobre todas las tablas y UPDATE sobre *categoria(tasa)*.

```

-- Rol comun.
CREATE ROLE empleado;
GRANT SELECT (apellido, nombre) ON cliente TO empleado;

-- Usuarios reales.
CREATE ROLE asesor          LOGIN PASSWORD 'pwd' IN ROLE empleado;
CREATE ROLE administrativo  LOGIN PASSWORD 'pwd' IN ROLE empleado;
CREATE ROLE encargado       LOGIN PASSWORD 'pwd' IN ROLE empleado;

GRANT SELECT ON taller TO asesor;
GRANT INSERT ON accidente TO asesor;
GRANT SELECT ON automovil TO administrativo;
GRANT UPDATE (tasa) ON categoria TO encargado;
-- DROP no es un privilegio que se otorgue: se concede ownership o
-- se ejecuta como propietario / superuser.

```

Ejercicio 4 (Oracle) — base de Accidentes

El Ejercicio 4 cierra el práctico ejercitando los aspectos más *Oracle*: rol explícito, perfil para acotar tiempo de conexión, PASSWORD EXPIRE y WITH GRANT OPTION a nivel de columnas.

```

-- Rol y privilegios comunes.
CREATE ROLE empleados;
GRANT SELECT ON automovil TO empleados;
GRANT SELECT ON categoria TO empleados;

-- Perfil para limitar al director.
CREATE PROFILE perfil_director LIMIT
    CONNECT_TIME 20;

-- Usuarios.
CREATE USER empleado1 IDENTIFIED BY pwd PASSWORD EXPIRE;
CREATE USER empleado2 IDENTIFIED BY pwd;
CREATE USER director IDENTIFIED BY pwd PROFILE perfil_director;

GRANT CONNECT TO empleado1, empleado2, director;
GRANT empleados TO empleado1, empleado2;

```

```
-- Privilegios específicos.
GRANT INSERT, UPDATE, DELETE ON accidente TO empleado1;
GRANT SELECT (apellido, nombre, direccion) ON cliente
  TO empleado2 WITH GRANT OPTION;

-- Director: SELECT sobre todo.
GRANT SELECT ON cliente TO director;
GRANT SELECT ON automovil TO director;
GRANT SELECT ON categoria TO director;
GRANT SELECT ON taller TO director;
GRANT SELECT ON accidente TO director;
```

11. Verificación de privilegios

Siempre que se asignan permisos hay que verificarlos. Cada motor tiene su forma:

- **MySQL:** `SHOW GRANTS FOR 'usuario'@'host';`
- **PostgreSQL:** las vistas de catálogo `information_schema.table_privileges`, `column_privileges` y `role_table_grants`; en `psql`, los meta-comandos `\dp` y `\z`.
- **Oracle:** las vistas `USER_TAB_PRIVS`, `USER_ROLE_PRIVS`, `USER_SYS_PRIVS` y, para los DBA, sus equivalentes `DBA_*`.

Una buena práctica del práctico es *loguearse efectivamente con cada usuario creado* y probar las operaciones permitidas (deben funcionar) y las prohibidas (deben fallar con error de privilegio). Si los *tests* positivos y negativos coinciden con la consigna, los permisos están bien definidos.

12. Ideas clave para repasar antes del parcial

1. **DCL es el sublenguaje de seguridad.** Sus dos comandos centrales son `GRANT` y `REVOKE`.
2. **Estándar vs implementación.** El estándar SQL define usuarios, roles y privilegios; la administración real la implementa cada motor. Hay que saber qué partes son comunes y qué partes cambian.
3. **Privilegios típicos:** `SELECT`, `INSERT`, `UPDATE`, `DELETE`, `REFERENCES`, `EXECUTE`, `CREATE`, `DROP`, `ALTER`, `INDEX`, `CREATE VIEW`.
4. **Granularidad:** los privilegios pueden ser globales (`*.*`), por base, por tabla o por columna. Para columnas se usa la lista entre paréntesis después del nombre del privilegio.
5. **WITH GRANT OPTION** habilita la *delegación*: el receptor puede a su vez otorgar el privilegio a otros.
6. **REVOKE ... CASCADE / RESTRICT** controla qué pasa con los privilegios delegados al revocar uno de ellos. Por defecto el comportamiento es en cascada.
7. **Roles** son “tipos de usuario”. Se crean con `CREATE ROLE`, se les otorgan privilegios con `GRANT ... ON ... TO role` y se asignan a usuarios con `GRANT role TO usuario`.
8. **El propietario** de un objeto es siempre quien lo creó. Sólo él (o un DBA) puede otorgar privilegios sobre él.
9. **Grafo de concesión:** existe un permiso si existe un camino desde la raíz (DBA o dueño) al usuario. Si al revocar se rompe el último camino, el usuario lo pierde.
10. **MySQL** maneja cuentas como `usuario@host` y permite limitar recursos con `MAX_QUERIES_PER_HOUR`, `MAX_CONNECTIONS_PER_HOUR`, etc.
11. **PostgreSQL** unifica usuarios, roles y grupos: todos son *roles*. `LOGIN` indica que pueden iniciar sesión.

12. **Oracle** asocia usuario y esquema; ofrece *perfiles* (`CREATE PROFILE`) para limitar recursos y `PASSWORD EXPIRE` para forzar cambio de clave.
13. **Verificar siempre:** `SHOW GRANTS` en MySQL, `\dp` en PostgreSQL, `USER_*_PRIVS` en Oracle. Y, además, probar conectándose como cada usuario.

Resumen final

DCL es la pieza que convierte una base de datos correctamente estructurada (DDL) y poblada (DML) en un sistema realmente *multiusuario* y seguro. La idea central es simple: cada operación debe estar respaldada por un privilegio explícito sobre el objeto correspondiente, y esos privilegios pueden viajar a través de roles (para escalar a muchos usuarios), de `WITH GRANT OPTION` (para delegar) y de un grafo de concesiones que ordena las relaciones entre quienes otorgan y reciben.

Si quedan claros los cinco bloques — usuarios, privilegios, roles, `WITH GRANT OPTION` con `REVOKE ... CASCADE/RESTRICT` y la administración por motor — el Práctico 2 se vuelve un mapeo directo de la teoría: cada inciso se traduce en una secuencia *crear usuario / otorgar privilegio / verificar / revocar*. El verdadero examen no es escribir el comando, sino justificar por qué ese conjunto de permisos cumple la consigna ni más ni menos que lo necesario.